Exercise 2: Orchestration with Kubernetes Using Minikube

Objective:
The objective of this exercise is to deploy a containerized C++ microservice on a local Kubernetes cluster using Minikube. The application is deployed using Kubernetes Deployment and exposed externally using a Kubernetes Service.

System Requirements

- Ubuntu-based Cloud Lab Environment
- Docker installed and running
- Minikube installed
- kubectl installed
- Web-based terminal access (No SSH required)

Prerequisite

Exercise 1 must be completed prior to this exercise. The Docker image of the C++ microservice should be available locally.

Architecture Overview

C++ HTTP Microservice → Docker Image → Kubernetes Deployment → Kubernetes Service → Exposed via Minikube URL

Step 1: Start Minikube Cluster

Minikube is started using Docker as the container runtime.

Command:
minikube start --driver=docker
Screenshot Placeholder:

```
student@student-virtual-machine:~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Excercise_1/cpp-docker$ minikube versi
on
minikube version: v1.38.0
commit: de81223c61ab1bd97dcfcfa6d9d5c59e5da4a0cf
student@student-virtual-machine:~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Excercise_1/cpp-docker$ minikube start
 --driver=docker
😄  minikube v1.38.0 on Ubuntu 22.04
✨  Using the docker driver based on user configuration
❗  Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
📌  Using Docker driver with root privileges
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.49 ...
💾  Downloading Kubernetes v1.35.0 preload ...
    > gcr.io/k8s-minikube/kicbase...:  514.16 MiB / 514.16 MiB  100.00% 39.31 M
    > preloaded-images-k8s-v18-v1...:  271.45 MiB / 271.45 MiB  100.00% 17.00 M
🔥  Creating docker container (CPUs=2, Memory=3900MB) ...
🐳  Preparing Kubernetes v1.35.0 on Docker 29.2.0 ...
🔗  Configuring bridge CNI (Container Networking Interface) ...
🔎  Verifying Kubernetes components...
    ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟  Enabled addons: storage-provisioner, default-storageclass
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
student@student-virtual-machine:~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Excercise_1/cpp-docker$
```

Step 2: Configure Docker Environment for Minikube

The Docker environment is configured to use Minikube's internal Docker daemon.

Command:
eval $(minikube docker-env)

Screenshot Placeholder:

```
student@student-virtual-machine:~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Excercise_1/cpp-docker$ kubectl get no
des
NAME       STATUS   ROLES           AGE    VERSION
minikube   Ready    control-plane   116s   v1.35.0
student@student-virtual-machine:~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Excercise_1/cpp-docker$
```

Step 3: Build Docker Image Inside Minikube

The Docker image for the C++ microservice is rebuilt inside Minikube.

Command:
docker build -t cpp-microservice .

Screenshot Placeholder:

```
student@student-virtual-machine:~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Excercise_1/cpp-docker$ kubectl get no
des
NAME       STATUS   ROLES           AGE    VERSION
minikube   Ready    control-plane   116s   v1.35.0
student@student-virtual-machine:~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Excercise_1/cpp-docker$ eval $(minikub
e docker-env)
student@student-virtual-machine:~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Excercise_1/cpp-docker$ docker build -
t cpp-microservice .
[+] Building 49.4s (10/10) FINISHED                                                                          docker:default
 => [internal] load build definition from Dockerfile                                                                  0.0s
 => => transferring dockerfile: 197B                                                                                  0.0s
 => [internal] load metadata for docker.io/library/ubuntu:22.04                                                       3.5s
 => [internal] load .dockerignore                                                                                     0.0s
 => => transferring context: 2B                                                                                       0.0s
 => [1/5] FROM docker.io/library/ubuntu:22.04@sha256:c7eb020043d8fc2ae0793fb35a37bff1cf33f156d4d4b12ccc7f3ef8706c38b1  5.1s
 => => resolve docker.io/library/ubuntu:22.04@sha256:c7eb020043d8fc2ae0793fb35a37bff1cf33f156d4d4b12ccc7f3ef8706c38b1  0.0s
 => => sha256:c7eb020043d8fc2ae0793fb35a37bff1cf33f156d4d4b12ccc7f3ef8706c38b1 6.69kB / 6.69kB                        0.0s
 => => sha256:34577a83cd7a8f1e55f6eb173fd9bb70c16127a83d6bf06f56f6b1fff9e486b9 424B / 424B                            0.0s
 => => sha256:65c77cbc27c2640de4c99ed5cfe6b689b192f81745a36032b50e2407b073ff22 2.30kB / 2.30kB                        0.0s
 => => sha256:6f4ebca3e823b18dac366f72e537b1772bc3522a5c7ae299d6491fb17378410e 29.54MB / 29.54MB                      1.9s
 => => extracting sha256:6f4ebca3e823b18dac366f72e537b1772bc3522a5c7ae299d6491fb17378410e                             2.9s
 => [internal] load build context                                                                                     0.0s
 => => transferring context: 804B                                                                                     0.0s
 => [2/5] RUN apt-get update && apt-get install -y g++                                                               36.7s
 => [3/5] WORKDIR /app                                                                                                0.0s
 => [4/5] COPY server.cpp .                                                                                           0.0s
 => [5/5] RUN g++ server.cpp -o server                                                                                1.0s
 => exporting to image                                                                                                3.0s
 => => exporting layers                                                                                               2.9s
 => => writing image sha256:a6a873ffb7e0fd4270b1056ac33afcb789123df3855eb6471977171530129f3a                          0.0s
 => => naming to docker.io/library/cpp-microservice                                                                   0.0s
student@student-virtual-machine:~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Excercise_1/cpp-docker$
```

```
student@student-virtual-machine:~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Excercise_1/cpp-docker$ docker images
                                                                                                    i Info →  U  In Use
IMAGE                                                ID             DISK USAGE   CONTENT SIZE   EXTRA
cpp-microservice:latest                              a6a873ffb7e0       381MB             0B
gcr.io/k8s-minikube/storage-provisioner:v5           6e38f40d628d      31.5MB             0B    U
registry.k8s.io/coredns/coredns:v1.13.1              aa5e3ebc0dfe      78.1MB             0B    U
registry.k8s.io/etcd:3.6.6-0                         0a108f718956      62.5MB             0B    U
registry.k8s.io/kube-apiserver:v1.35.0               5c6acd67e9cd      89.8MB             0B    U
registry.k8s.io/kube-controller-manager:v1.35.0      2c9a4b058bd7      75.8MB             0B    U
registry.k8s.io/kube-proxy:v1.35.0                   32652ff1bbe6      70.7MB             0B    U
registry.k8s.io/kube-scheduler:v1.35.0               550794e3b12a      51.7MB             0B    U
registry.k8s.io/pause:3.10.1                         cd073f4c5f6a       736kB             0B    U
student@student-virtual-machine:~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Excercise_1/cpp-docker$
```

Step 4: Create Kubernetes Deployment

A Kubernetes Deployment YAML file is created to manage the C++ microservice pods.

File Name: deployment.yaml

Screenshot Placeholder:

```
student@student-virtual-machine: ~/25SUB4508_LSP/25SUB4508_56133/As...   student@student-virtual-machine: ~/25SUB4508_LSP/25SUB4508_56133/As...
 GNU nano 6.2                                        deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cpp-microservice-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: cpp-microservice
  template:
    metadata:
      labels:
        app: cpp-microservice
    spec:
      containers:
      - name: cpp-container
        image: cpp-microservice
        imagePullPolicy: Never
        ports:
        - containerPort: 8080
```

Step 5: Apply Deployment Configuration

The deployment configuration is applied using kubectl.

Command:
kubectl apply -f deployment.yaml

Step 6: Verify Deployment

The status of the deployment is verified.

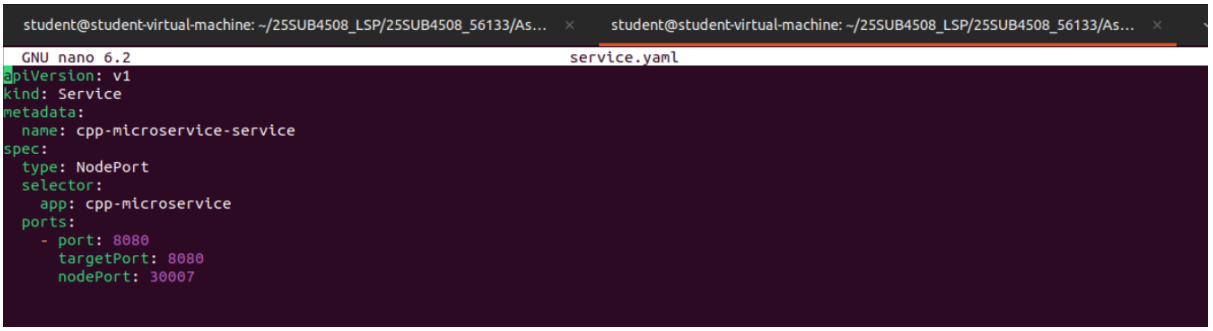
Commands:
kubectl get deployments

Step 7: Create Kubernetes Service

A Kubernetes Service is created to expose the microservice externally.

File Name: service.yaml

Screenshot Placeholder:



Step 8: Apply Service Configuration

The service configuration is applied using kubectl.

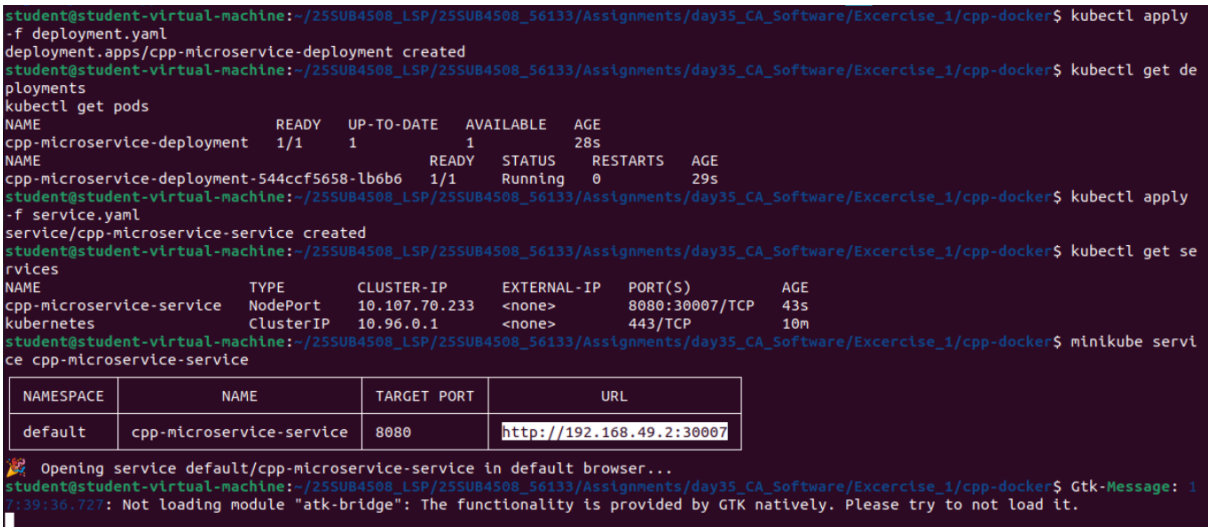
Command:
kubectl apply -f service.yaml

Step 9: Access Microservice Using Minikube

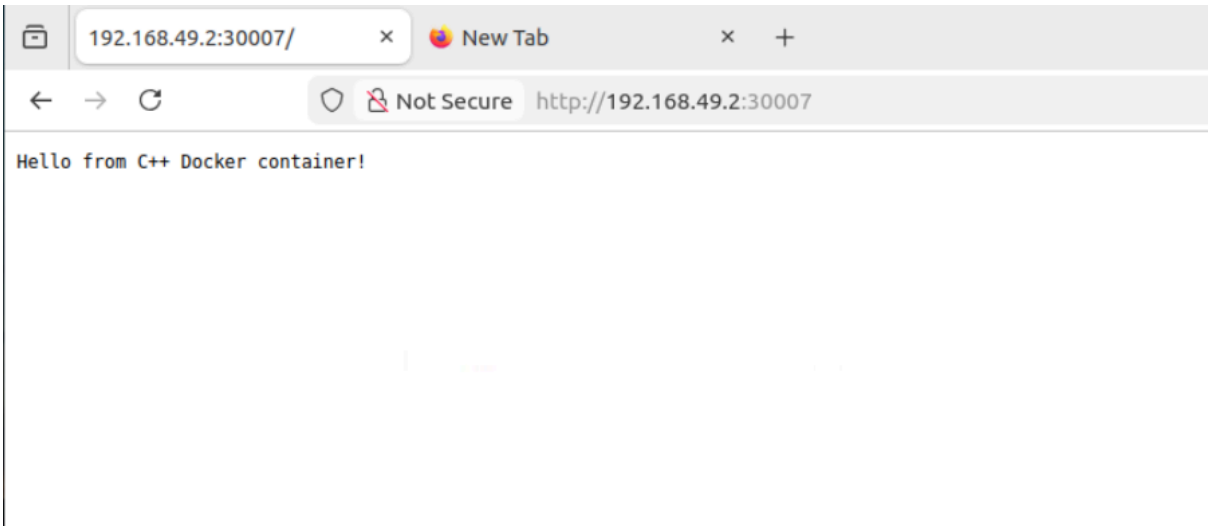
The microservice is accessed using the Minikube service URL.

Command:
minikube service cpp-microservice-service

Screenshot Placeholder:



Screenshot Placeholder:



Result

The containerized C++ microservice was successfully deployed on a local Kubernetes cluster using Minikube. The application was exposed using a Kubernetes Service and accessed externally.

Conclusion

This exercise demonstrates Kubernetes orchestration concepts including deployments, services, and cluster management using Minikube. The microservice deployment was performed without SSH access, making it suitable for cloud lab environments.